Roll No: 2025115054

1) Implement a Stack using Queue data structure.

Program:

```
#include <stdio.h>
#define MAX 20

int q1[MAX], q2[MAX];
int f1 = 0, r1 = 0, f2 = 0, r2 = 0;

void push(int x)
{
    while (f1 < r1)
        q2[r2++] = q1[f1++];

    q1[r1++] = x;

    while (f2 < r2)
        q1[r1++] = q2[f2++];

    f1 = 0; r2 = 0; f2 = 0;
}

int pop()
{
    if (f1 == r1)
        return -1;
    return q1[f1++];
}

void display()
{
    int i;
    for (i = f1; i < r1; i++)
        printf("%d ", q1[i]);
}

int main()
{
    push(10);
    push(20);
    push(30);

    printf("Stack elements: ");
    display();

    printf("\nPopped element: %d", pop());
    printf("\nStack after pop: ");
    display();

    return 0;
}
```

Output:

```
Stack elements: 30 20 10
Popped element: 30
Stack after pop: 20 10
```

2) Implement a Queue using Stack data structure.

Program:

```
#include <stdio.h>
#define MAX 20

int s1[MAX], s2[MAX];
int top1 = -1, top2 = -1;

void enqueue(int x)
{
    s1[++top1] = x;
}

int dequeue()
{
    int x;

    if (top1 == -1 && top2 == -1)
        return -1;

    if (top2 == -1)
        while (top1 != -1)
            s2[++top2] = s1[top1--];

    x = s2[top2--];
    return x;
}

void display()
{
    int i;
    if (top2 != -1)
        for (i = top2; i >= 0; i--)
            printf("%d ", s2[i]);
    for (i = 0; i <= top1; i++)
        printf("%d ", s1[i]);
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    printf("Queue elements: ");
    display();

    printf("\nDeleted element: %d", dequeue());
    printf("\nQueue after deletion: ");
    display();

    return 0;
}
```

Output:

```
Queue elements: 10 20 30
Deleted element: 10
Queue after deletion: 20 30
```

3) Write a C program using stack data structure to manage the history of web pages visited.

Program:

```
#include <stdio.h>
#include <string.h>
#define MAX 20

char back[MAX][50], forward[MAX][50];
int topBack = -1, topForward = -1;
char current[50] = "Home";

void visit(char page[])
{
    strcpy(back[++topBack], current);
    strcpy(current, page);
    topForward = -1;
}

void goBack()
{
    if (topBack == -1)
        printf("\nNo previous page.");
    else
    {
        strcpy(forward[++topForward], current);
        strcpy(current, back[topBack--]);
    }
}

void goForward()
{
    if (topForward == -1)
        printf("\nNo forward page.");
    else
    {
        strcpy(back[++topBack], current);
        strcpy(current, forward[topForward--]);
    }
}

int main()
{
    visit("Google");
    visit("YouTube");
    visit("Wikipedia");

    printf("Current page: %s", current);

    goBack();
    printf("\nAfter Back: %s", current);

    goBack();
    printf("\nAfter Back: %s", current);

    goForward();
    printf("\nAfter Forward: %s", current);

    return 0;
}
```

Output:

```
Current page: Wikipedia
After Back: YouTube
After Back: Google
After Forward: YouTube
```

4) Write a C program to sort a stack using temporary stack.

Program:

```
#include <stdio.h>
#define MAX 20

int main()
{
    int stack[MAX], temp[MAX];
    int top = -1, ttop = -1;
    int n, i, x;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter stack elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &stack[++top]);

    while (top != -1)
    {
        x = stack[top--];

        while (ttop != -1 && temp[ttop] > x)
            stack[++top] = temp[ttop--];

        temp[++ttop] = x;
    }

    printf("Sorted stack (top to bottom): ");
    while (ttop != -1)
        printf("%d ", temp[ttop--]);

    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter stack elements: 34 12 56 8 25 19
Sorted stack (top to bottom): 8 12 19 25 34 56
```

5) The Stock Span Problem: Given daily stock prices, calculate the span of the stock's price for each day using a stack. The span is the maximum number of consecutive days leading up to the current day where the stock price is less than or equal to the current day's price.

Program:

```
#include <stdio.h>
#define MAX 50

int main()
{
    int price[MAX], span[MAX], stack[MAX];
    int top = -1, n, i;

    printf("Enter number of days: ");
    scanf("%d", &n);

    printf("Enter stock prices: ");
    for (i = 0; i < n; i++)
        scanf("%d", &price[i]);

    for (i = 0; i < n; i++)
    {
        while (top != -1 && price[stack[top]] <= price[i])
            top--;

        if (top == -1)
            span[i] = i + 1;
        else
            span[i] = i - stack[top];

        stack[++top] = i;
    }

    printf("Stock spans: ");
    for (i = 0; i < n; i++)
        printf("%d ", span[i]);

    return 0;
}
```

Output:

```
Enter number of days: 8
Enter stock prices: 100 80 60 70 60 75 85 90
Stock spans: 1 1 1 2 1 4 6 7
```

6) Given an array, for each element find the nearest element to the right that has a frequency greater than the current element's frequency. If no such element exists, return -1. Implement using stack.

Program:

```
#include <stdio.h>
#define MAX 50

int main()
{
    int a[MAX], freq[MAX], ans[MAX], stack[MAX];
    int n, i, j, top = -1;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n; i++)
    {
        freq[i] = 0;
        for (j = 0; j < n; j++)
            if (a[i] == a[j])
                freq[i]++;
    }

    for (i = n - 1; i >= 0; i--)
    {
        while (top != -1 && freq[stack[top]] <= freq[i])
            top--;

        if (top == -1)
            ans[i] = -1;
        else
            ans[i] = a[stack[top]];

        stack[++top] = i;
    }

    printf("Answer: ");
    for (i = 0; i < n; i++)
        printf("%d ", ans[i]);

    return 0;
}
```

Output:

```
Enter number of elements: 6
Enter elements: 2 1 1 3 2 1
Answer: 1 -1 -1 2 1 -1
```

7) Given a pattern containing only I's and D's, where I is for increasing and D is for decreasing, write an algorithm using stack to print the minimum number following that pattern. Digits are from 1-9 and digits cannot repeat.

Program:

```
#include <stdio.h>
#include <string.h>
int main()
{
    char pattern[20];
    int stack[20], top = -1;
    int i, num = 1;
    printf("Enter pattern: ");
    scanf("%s", pattern);
    for (i = 0; i <= (int)strlen(pattern); i++)
    {
        stack[++top] = num++;
        if (i == (int)strlen(pattern) || pattern[i] == 'I')
        {
            while (top != -1)
                printf("%d", stack[top--]);
        }
    }
    return 0;
}
```

Output:

```
Enter pattern: D
21

Enter pattern: DIDD
21435
```

8) Given an array of digits (values are from 0 to 9), find the minimum possible sum of two numbers formed from digits of the array. All digits of the given array must be used to form the two numbers. Use Queue for implementation.

Program:

```
#include <stdio.h>
#define MAX 50
int main()
{
    int queue[MAX], n, i, j, temp;
    int front = 0, rear = -1;
    int num1 = 0, num2 = 0;
    printf("Enter number of digits: ");
    scanf("%d", &n);
    printf("Enter digits: ");
    for (i = 0; i < n; i++)
        scanf("%d", &queue[++rear]);

    /* Sort the queue elements */
    for (i = 0; i < n - 1; i++)
        for (j = i + 1; j < n; j++)
            if (queue[i] > queue[j])
            {
                temp = queue[i];
                queue[i] = queue[j];
                queue[j] = temp;
            }

    /* Dequeue alternately to form two numbers */
    for (i = 0; i < n; i++)
    {
        if (i % 2 == 0)
            num1 = num1 * 10 + queue[front++];
        else
            num2 = num2 * 10 + queue[front++];
    }

    printf("Minimum sum = %d", num1 + num2);
    printf("\\nNumbers formed = %d and %d", num1, num2);
    return 0;
}
```

Output:

```
Enter number of digits: 6
Enter digits: 6 8 4 5 2 3
Minimum sum = 604
Numbers formed = 246 and 358
```

9) Given an array and a positive integer K, find the first negative integer for each contiguous subarray of size K. If all integers in the window are positive, print 0.

Program:

```
#include <stdio.h>
#define MAX 50

int main()
{
    int a[MAX], q[MAX], n, k, i;
    int front = 0, rear = -1;

    printf("Enter array size: ");
    scanf("%d", &n);

    printf("Enter array elements: ");
    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Enter window size: ");
    scanf("%d", &k);

    printf("First negative values: ");

    for (i = 0; i < n; i++)
    {
        if (a[i] < 0)
            q[++rear] = i;

        if (i >= k - 1)
        {
            while (front <= rear && q[front] < i - k + 1)
                front++;

            if (front <= rear)
                printf("%d ", a[q[front]]);
            else
                printf("0 ");
        }
    }

    return 0;
}
```

Output:

```
Enter array size: 5
Enter array elements: -8 2 3 -6 10
Enter window size: 2
First negative values: -8 0 -6 -6
```

10) Given a matrix of dimension m x n where each cell can have value 0, 1, or 2: 0 = empty cell, 1 = cell having fresh oranges, 2 = cell having rotten oranges. Find the minimum time required so that all oranges become rotten. A rotten orange can rot fresh oranges in its neighbouring cells (up, down, left and right). Use Queue for implementation.

Program:

```
#include <stdio.h>
#define MAX 20

struct Cell
{
    int r, c, time;
};

int main()
{
    int a[MAX][MAX], m, n, i, j;
    int front = 0, rear = -1, fresh = 0, time = 0;
    struct Cell q[MAX * MAX];

    printf("Enter rows and columns: ");
    scanf("%d %d", &m, &n);

    printf("Enter matrix:\n");
    for (i = 0; i < m; i++)
        for (j = 0; j < n; j++)
        {
            scanf("%d", &a[i][j]);

            if (a[i][j] == 1)
                fresh++;

            if (a[i][j] == 2)
            {
                q[++rear].r = i;
                q[rear].c = j;
                q[rear].time = 0;
            }
        }

    while (front <= rear)
    {
        struct Cell cur = q[front++];
        int dr[4] = {-1, 1, 0, 0};
        int dc[4] = {0, 0, -1, 1};

        if (cur.time > time)
            time = cur.time;

        for (i = 0; i < 4; i++)
        {
            int nr = cur.r + dr[i];
            int nc = cur.c + dc[i];

            if (nr >= 0 && nr < m && nc >= 0 && nc < n &&
                a[nr][nc] == 1)
            {
                a[nr][nc] = 2;
                fresh--;

                q[++rear].r = nr;
                q[rear].c = nc;
                q[rear].time = cur.time + 1;
            }
        }
    }

    if (fresh == 0)
        printf("Minimum time = %d", time);
    else
        printf("All oranges cannot be rotten.");

    return 0;
}
```

Output:

```
Enter rows and columns: 3 3
Enter matrix:
2 1 0
1 0 2
0 0 2
Minimum time = 1
```